Ciudad interactiva con OpenGL



NOMBRE: JOSUE DAVID TORREZ DURAN

REGISTRO: 222010045

Ciudad interactiva con OpenGL

Explicación de mi proyecto y de los cambios que realicé.

1. ¿Qué cambié en la ciudad?

Básicamente, amplié la ciudad de 7 × 7 a 11 × 11 celdas. Mantuve el tamaño de cada celda en 10 unidades, así que ahora el mapa mide 110 × 110 unidades. En total tiene 21 manzanas con edificios y 4 parques. Los edificios tienen diferentes alturas y colores, y las calles conectan los distintos sectores.

Para definir la ciudad uso una matriz llamada MAPA. Cada número indica lo que hay en esa parte: 0 es calle, 1 es edificio y 2 es parque. Con esa matriz, el programa recorre cada posición y dibuja lo que corresponde.

También adapté el suelo, los límites, las colisiones, la cámara general y el minimapa. De esta forma puedo conducir por las calles nuevas, mientras que los edificios, los parques y los bordes siguen bloqueando el paso del automóvil.

2. ¿Cómo está organizado el código?

Mantuve el proyecto dividido en cuatro clases. Cada una agrega una parte, así que puedo ubicar más fácilmente dónde está cada función.

Archivo	Qué hace
clase1.java	Crea la ciudad, define MAPA y dibuja el suelo, las calles y los edificios. También prepara OpenGL y la cámara general.
clase2.java	Agrega el automóvil, su movimiento, la velocidad, los giros, las colisiones y la cámara que lo sigue.
clase3.java	Agrega el cambio entre día y noche, las farolas y los dos faros delanteros. Aquí está el cálculo de la iluminación.
clase4.java	Agrega la decoración, los semáforos y el minimapa. La versión del proyecto también conserva las entregas que ya tenía.

3. Iluminación y decoración

Distribuí 25 farolas por la ciudad. Su luz ilumina el suelo y las superficies cercanas, y pierde intensidad con la distancia. Los dos faros delanteros acompañan la posición y la dirección del automóvil; su iluminación termina a 24 unidades.

Para calcular la luz, el programa toma en cuenta hacia dónde mira cada superficie mediante sus normales. Por separado, hace que las bombillas y las ventanas se vean encendidas de noche. Los parques tienen árboles y bancos, y junto a las manzanas hay pasos peatonales y semáforos.

4. ¿Cómo funciona la conducción?

El movimiento usa el tiempo que pasa entre cuadros. Así, el desplazamiento se calcula en función de los segundos y no simplemente de cuántas veces se dibuja la escena. Para las colisiones, el programa representa al automóvil con un círculo de radio 1,65 y comprueba que no invada las aceras ni salga del mapa.

El minimapa muestra toda la ciudad desde arriba, con el norte hacia la parte superior. También marca la posición y la orientación del automóvil. Al cambiar el tamaño de la ventana, el recuadro se ajusta; su dibujo se limita a esa esquina para conservar la vista principal.

Los semáforos repiten un ciclo de rojo durante 5 segundos, verde durante 5 segundos y amarillo durante 2 segundos. Son una función visual: no obligan al automóvil a detenerse.

5. Controles

Tecla	Acción
W / S o arriba / abajo	Acelerar / frenar con motor y retroceder.
A / D o izquierda / derecha	Girar mientras el automóvil se mueve.
Espacio	Frenar.
C	Cambiar entre la cámara general y la que sigue al automóvil.
N / F / M	Cambiar día-noche / encender o apagar faros / mostrar u ocultar el mapa.
R / Esc	Reiniciar el automóvil y el progreso / salir.

6. ¿Cómo lo ejecuto?

Necesito Java JDK 17 o superior y compatibilidad con OpenGL 3.3. En IntelliJ IDEA abro la carpeta que contiene pom.xml, espero a que Maven cargue las dependencias y abro clase4.java. Después presiono el triángulo que aparece junto a main y selecciono Run clase4.main().

Si lo ejecuto desde una terminal, uso este comando dentro de la carpeta del proyecto:

```
mvn clean compile exec:exec
```

La primera compilación necesita conexión para descargar las dependencias. En macOS, la ejecución directa desde el editor requiere la opción -XstartOnFirstThread; al usar Maven, el perfil del proyecto ya la agrega.

7. Si me piden cambiar el tamaño, ¿dónde lo hago?

La parte principal está en clase1.java, al inicio del archivo, en la sección 1: VARIABLES DE LA APLICACIÓN. Ahí encuentro CELDA, la matriz MAPA y LIMITE. En IntelliJ puedo presionar Ctrl + F y buscar MAPA para llegar directamente.

```
protected static final float CELDA = 10;
protected static final int[][] MAPA = { /* filas del mapa */ };
protected static final float LIMITE = MAPA.length * CELDA / 2;
```

Si me piden pasar de 11 × 11 a 13 × 13, cambio MAPA para que tenga 13 filas y 13 números en cada fila. Mantengo CELDA en 10 para conservar el tamaño de las calles y las manzanas. Con el patrón actual, dejo las filas y columnas de índice par como calles y las posiciones impares como parcelas.

LIMITE se calcula automáticamente con la cantidad de filas. Con 13 × 13 pasa de 55 a 65, por lo que el mapa mide 130 × 130 unidades. El suelo, los centros de las celdas, los límites de conducción, la cámara general y el minimapa usan ese valor.

También debo revisar estas partes para que el cambio quede completo:

Dónde	Qué cambio para 13 × 13
clase3.java, crearFarolas()	Cambio new float[25][3] por new float[36][3], porque con este patrón habrá 6 × 6 farolas.
clase3.java, fragmentShader()	Cambio uLuces[25] por uLuces[36] y el límite del bucle indice < 25 por indice < 36.
clase4.java, DESTINOS	Si mantengo las entregas, cambio centro(10) por centro(MAPA.length - 1) para que las metas queden en el nuevo borde.
ConduccionTest.java	Actualizo las dimensiones, las cantidades y las coordenadas que esperan las pruebas.

La posición inicial del automóvil ya usa centro(0) y centro(MAPA.length - 1), por lo que se ajusta al nuevo mapa. Para una matriz impar N × N con este patrón, la cantidad de farolas es ((N - 1) / 2)².

Si me piden cambiar el tamaño de la ventana, cambio ancho = 1100 y alto = 760, también al inicio de clase1.java. Eso modifica la ventana, pero no la ciudad. Cambiar CELDA requiere revisar además tamaños de edificios, aceras, colisiones, decoración y luces; por eso para ampliar el mapa mantengo CELDA = 10.

8. Comprobaciones y limitaciones

El código compiló y pasaron tres pruebas de distribución y conexión de calles, colisiones y entregas/reinicio. También se comprobó que los shaders compilan y se enlazan en OpenGL 3.3 mediante Mesa/EGL. La ventana gráfica todavía debe probarse manualmente para confirmar el teclado, las cámaras y el redimensionamiento.

Para mostrar que la ampliación funciona, puedo conducir por las calles nuevas, intentar entrar en una manzana y acercarme a los bordes. Después puedo cambiar la cámara, el día y la noche, los faros y el minimapa. Con R compruebo que el automóvil vuelve a su posición inicial, recupera su orientación y queda detenido.

La iluminación no genera sombras ni calcula si otro edificio bloquea una fuente de luz. Tampoco hay tráfico autónomo ni un panel gráfico dentro de la ventana; la velocidad y los demás indicadores aparecen en el título. Los modelos se construyen con cajas para mantener el proyecto sencillo.

Según la aclaración del docente, las metas o checkpoints son opcionales. La versión entregada ya los conserva. Este documento explica el proyecto y no cambia esas funciones.

9. Recursos utilizados

Tomé como base el proyecto CiudadConduccion proporcionado para la actividad y adapté sus clases. La geometría se genera mediante código; no agregué modelos ni texturas externas. Las dependencias están declaradas en pom.xml: LWJGL 3.3.3 para GLFW y OpenGL, JUnit para las pruebas y los plugins de Maven.